

1 Word Embeddings (Lec 5)

```
def euclidean_distance(a, b): # Euclidean distance: smaller distance - more similar
    return np.linalg.norm(a - b)
def dot_product_similarity(a, b): # Dot product similarity: larger dot product - more similar
    return np.dot(a, b)
def cosine_similarity(a, b): # Cosine similarity: larger similarity score - more similar
    return dot(a, b) / (norm(a) * norm(b))
v1 = np.array([2, 4, 3])
v2 = np.array([5, 1, 0])
print(f"Euclidean Distance: {euclidean_distance(v1, v2):.4f}")
print(f"Dot Product Similarity: {dot_product_similarity(v1, v2):.4f}")
print(f"Cosine Similarity: {cosine_similarity(v1, v2):.4f}")
```

Euclidean Distance Similarity:— Rank 1 (Most Similar): Item 1. The distance between the tip of the "Query" vector and "item 1" is the smallest. **Rank 2: Item 3.** The distance to the tip of "item 3" is significantly larger than item 1. **Rank 3 (Least Similar):** Item 2. Despite having a similar angle to item 3, its extreme magnitude places its endpoint furthest from the query. **Dot Product Similarity:— Rank 1 (Most Similar): Item 2.** Although the angle θ is larger than item 1, the massive magnitude $\|B\|$ of item 2 dominates the product. **Rank 2: Item 3.** It has a similar angle to item 2 but a smaller magnitude, resulting in a lower product. **Rank 3 (Least Similar): Item 1.** Despite having the smallest angle θ (highest $\cos(\theta)$), its negligible magnitude results in the smallest dot product. **Cosine Similarity:— Rank 1 (Most Similar): Item 1.** The angular displacement between "Query" and "item 1" is the smallest, meaning $\cos(\theta)$ is closest to 1. **Rank 2: Item 2 / Item 3 (Tie).** Visually, item 2 and item 3 appear to lie on the same ray or have very similar angular offsets from the query. If item 3 is slightly closer in angle than item 2, it would take Rank 2. **Rank 3 (Least Similar): Item 2 / Item 3.** (The one with the larger angular gap).

1.1 Vector Space Models

- Map words to dense vectors where geometric distance $\approx$ semantic similarity.
- **One-hot:** $|V|$ dimensions. Sparse. No inherent notion of relationship between words and the dot product between similar and non-similar words is zero.
 - **Term-Term Co-occurrence:** go through a corpus of text, keeping a count of all of the words that appear in context of each word (within a window). The similarity is calculated using dot products between word vectors.
 - **LSA** count-based/matrix factorization. LSA can generate dense word embeddings. LSA can generate dense word embeddings.
 - **Word2Vec** Continuous bag of words (CBOW), Skip-gram optimized by Hierarchical softmax and Negative sampling.

1.2 Word2Vec - Skip Gram - Gensim

Dense Representations. NN model to obtain short and dense representations of words. A simple architecture with 1. Input layer 2. linear hidden layer (without any activation function) 3. Output layer with softmax. Goal is learn meaningful weights (meaningful representation of the input) in the process. Given a target word (i.e., center word) word, predict context words (i.e., surrounding words). **Negative sampling:** Instead of computing probabilities for all words, the model updates only a small number of negative samples (randomly selected words that are unlikely to be in the context). **Hierarchical softmax:** Instead of computing softmax over the entire vocabulary, words are arranged in a binary tree structure, allowing predictions to be made in $\log(V)$ time instead of $O(V)$. **Pre-trained Embeddings word2vec** trained on several corpora using the word2vec algorithm. **fastText** **Implicit biases and stereotypes in word embeddings:** Embeddings reflect gender stereotypes present in broader society. They may also amplify these stereotypes because of their widespread usage.

```
google_news_vectors = api.load("word2vec-google-news-300")
model = Word2Vec(sentences, vector_size=10, window=5, min_count=1) # train word2vec model
model.wv.most_similar("cat")
```

The Skip-gram Model Example

Add freshly **squeezed** context, **pineapple** target, **juice** context to your smoothie .

Size of the vocabulary $V = 10,000$
Size of the embedding $d = 300$
Window size 1

Parameters of the model

Input x

0	0.02	-0.13	-0.28	0.13	0.42	-0.04	-0.03	0.19	0.011	0
1	-0.12	-0.03	0.22	0.03	0.3	-0.5	0.16	-0.28	0.21	0
2	0.1	0.1	0.1	0.1	0.06	-0.93	0.11	2.3	0.002	0
3	0.4	-0.2	0.05	-0.2				-0.02		0

One-hot encoding of the word **pineapple**

Word embedding matrix

Select the column of W corresponding to the target word **pineapple**

Shared context matrix

Output layer output $W^o W_x$

-0.19	0
-0.28	0
2.3	0
-0.02	0

Prediction

Softmax layer output

0	0
1	1
2	0
3	0

Prob. dist of $W^o W_x$

One-hot encoding of the word **juice**

2 Latent Semantic Analysis LSA $X_{n \times d} \approx Z_{n \times k} W_{k \times d}$

n number of documents, k number of topics, d number of features, Z is document topic assignments, W is topic-word assignments. LSA Truncated SVD for matrix factorization using SVD (without centering the data). **Advantage:** The idea is to reduce the dimensionality of the data to ex-

tract some semantically meaningful components. **Disadvantage:** Topics are not probability distributions over words. Documents are not explicitly modeled as mixtures of topics. There is no generative process explaining how words arise. Not Probabilistic generative model hence we use LDA.

3 Latent Dirichlet Allocation LDA

LSA disadvantage is **advantage**. we assume each document has a distribution over topics and each topic has a distribution over words. LDA is a Bayesian probabilistic generative model with Dirichlet priors, we compute the posterior distribution over these latent variables. However, this posterior has no closed-form solution. So we rely on approximate inference methods. **how dif from k-means:** In clustering with K-Means, documents are grouped based on a similarity measure, and each document is assigned to a single cluster. In contrast, topic modeling produces distributions over words for each topic, as well as distributions over topics per document. Topic modeling can be thought of as a form of soft clustering, where documents are assigned to multiple topics based on their probability distributions rather than a single cluster. The Dirichlet controls how mixed or concentrated a probability distribution is. **Case 1: Small $\alpha=0.1$** - Distributions are sparse and peaked. A document is mostly about one topic. Topics are specialized. Fewer mixtures. **Case 2: Large $\alpha=10$** - Distributions are smooth and spread out. A document mixes many topics evenly. Topics are less distinct. More mixing. **Gibbs sampling (accurate but slow; good for small datasets)** Markov Chain Monte Carlo (MCMC) algorithm used to estimate the posterior distribution of hidden variables (topic assignments) that are otherwise computationally intractable to calculate directly. **3.1 Topic Modeling Pipeline** **1. Corpus Preprocessing:** Normalization via case-folding and punctuation removal. Token filtration involves n -length thresholds and frequency pruning (e.g., **filter_extremes**). **Lemmatization** reduces inflected forms to morphological roots; **POS tagging** can restrict the feature space to specific grammatical categories (e.g., Nouns/Adjectives) to enhance semantic density. **2. Model Training (Gensim/sklearn):** Construction of a Document-Term Matrix (DTM). Using **corpora.Dictionary**, tokens are mapped to indices (**token2id**) and documents are transformed into sparse **Bag-of-Words (BoW)** vectors (**doc2bow**). Key hyperparameters: K (**num_topics**) and **passes** (epochs). **3. Evaluation & Interpretation:** **• Qualitative Analysis:** "Eyeballing" top- N words per topic or documents with high posterior probability $P(z|d)$. **• Human Judgments:** Assessing "story coherence" or **Word Intrusion** tests. In intrusion, an "outlier" word from a distal topic is inserted into a top-word set; model quality is measured by the human ability to identify the intruder (e.g., *dentist*, *appointment*, *mom*, *doctor*). **• Extrinsic Evaluation:** Benchmarking topics as downstream features (e.g., in Sentiment Analysis) to see if they improve predictive accuracy. **• Quantitative Metrics (Coherence):** **CoherenceModel** measures semantic similarity between the most probable words in a topic. Scores range from $[-1, 1]$, where higher values indicate stronger semantic interpretability. **4 Recommender Systems: Introduction** Recommender systems mitigate **information overload** by filtering large item spaces to suggest products/services based on predicted consumption. They optimize the discovery process by mapping user preferences to item attributes or behavioral patterns. **4.1 Taxonomy of Recommenders** **• Collaborative Filtering (CF):** Unsupervised approach leveraging user-item interaction matrices to discover **latent features**. **• Content-based:** Supervised learning utilizing explicit user/item metadata (e.g., genre, demographics) to predict affinities. **• Hybrid:** Ensemble methods combining CF and Content-based signals to mitigate individual algorithmic weaknesses. **4.2 The Filter Bubble & Ethics** Exploitation-heavy algorithms lead to **Filter Bubbles**, narrowing user exposure to diverse content. Ethical risks include the "Cold Start" problem (new users/items), data privacy concerns, and the potential for maximizing engagement at the cost of radicalization or misinformation. **5 Problem Formulation & Baselines** **5.1 The Utility Matrix (Y)** An $N \times M$ matrix where N = users and M = items. Y is typically **highly sparse** ($< 1\%$ density). The objective is **Matrix Completion:** predicting unobserved entries $\hat{y}_{ij}$ to recommend high-value items. **Evaluation Metrics** Performance is validated via (RMSE) or (MSE). **5.2 Baseline Imputation** **• Simple Averages:** Global μ , per-user μ_i , or per-item μ_j . **• K-Nearest Neighbors (KNN):** Imputing missing values based on the k most similar users/items in the training manifold.

```
from sklearn.impute import KNNImputer
imputer = KNNImputer(n_neighbors=2) # Impute missing values based on spatial proximity
```

```
train_mat_imp = imputer.fit_transform(train_mat)
```

6 Collaborative Filtering & SVD

6.1 Matrix Factorization (MF)

Approximates $Y \approx ZW^T$, where $Z \in \mathbb{R}^{N \times k}$ and $W \in \mathbb{R}^{M \times k}$. **• z_i (User Embedding):** Latent representation of user i . **• w_j (Item Embedding):** Latent representation of item j . The predicted rating is the dot product: $\hat{y}_{ij} = w_j^T z_i$. **Regularized Objective Function:** Since standard SVD is undefined for missing data, CF minimizes the squared error over **observed ratings** R with L_2 regularization: $f(Z, W) = \sum_{(i,j) \in R} (w_j^T z_i - y_{ij})^2 + \frac{\lambda}{2} \|Z\|_F^2 + \frac{\lambda}{2} \|W\|_F^2$ **Incorporating Biases:** To account for systematic variations (e.g., "easy" raters or "popular" items), we add bias terms: $\hat{y}_{ij} = \beta + \beta_i + \beta_j + w_j^T z_i$ Where β is global mean, β_i is user bias, and β_j is item bias.

7 Implementation: Surprise Library

The **surprise** library uses Stochastic Gradient Descent (SGD) to minimize the regularized loss for SVD models.

```
from surprise import SVD, Dataset, Reader
# Define scale and load data
reader = Reader(rating_scale=(1, 5))
data = Dataset.load_from_df(df, reader)
# Initialize SVD with k latent factors
algo = SVD(n_factors=2)
trainset = data.build_full_trainset()
algo.fit(trainset)
```

7.1 Problem Definition

The objective is to suggest items based on latent preference. Formulated as a **matrix completion** problem using a utility matrix $Y_{N \times M}$ encoding interactions between N users and M items. Y is typically sparse ($< 1\%$ density).

7.2 Approaches

- **Collaborative Filtering (CF):** Leverages social signals/past agreement. Maps users/items to latent factor spaces Z and W .
- **Content-based Filtering:** Supervised learning using intrinsic item features to predict ratings.
- **Hybrid:** Weighted ensembles or cascaded models combining CF and content signals.

7.3 Dot Product vs. Cosine Similarity

Dot Product ($u \cdot v = \|u\| \|v\| \cos \theta$) is sensitive to vector magnitude, favoring "popular" items with high-norm embeddings. **Cosine Similarity** ($\frac{u \cdot v}{\|u\| \|v\|}$) normalizes by magnitude, measuring only angular displacement. This improves diversity by reducing popularity bias.

8 Content-Based Filtering

8.1 Methodology

Construct user profiles based on historical item features. For each user i , train a regressor (e.g., **Ridge**) where X is the feature matrix and y are the user's ratings.

8.2 Pros and Cons

Pros: No item cold-start problem as brand-new movies can be recommended using metadata (genres, actors, etc.) even with zero ratings; high transparency and interpretability because recommendations are easily explained via specific feature weights; and user independence, ensuring that poor or biased data from other users does not impact an individual's profile. **Cons:** Significant user cold-start problem where users with sparse histories provide insufficient data for reliable modeling; low serendipity as the system lacks the ability to suggest diverse genres, instead recommending "more of the same"; and heavy dependency on high-quality metadata, as inaccurate or broad tags limit the model's predictive power

9 Implementation at Scale

9.1 Sparse Representations

Utility matrices for large sets are memory-prohibitive in dense formats. Using Compressed Sparse Row (**CSR**) formats stores only non-zero entries, reducing memory (e.g., 600GB dense $\rightarrow$ 8MB sparse).

9.2 Data Modalities

Explicit: Direct feedback (ratings/likes). **Implicit:** Behavioral proxies (dwell time, click-through, purchase). Implicit data is more abundant but noisier.

9.3 Evaluation Limitations

RMSE and MAE measure prediction accuracy but fail to capture user satisfaction. There is no single "ground truth" for preference.

9.4 Optimization Dimensions

- **Diversity:** Variance in recommendation categories.
- **Freshness/Serendipity:** Surprising or novel items.
- **Trust/Explainability:** Providing rationale for the suggestion.
- **Persistence:** Handling of ignored or decayed recommendations.

10 Ethics and Consequences

Algorithms optimizing solely for engagement can catalyze extremist rabbit holes and political polarization. Designers must evaluate the societal impact and "Filter Bubble" effects before deployment.